

06 - Arquitectura de la aplicación

Ejercicio 06 — Arquitectura de la aplicación



Foco: arquitectura, diseño en capas, decisiones de diseño **Tiempo estimado:** 2 clases [↩ Volver al README](#)

Objetivo

Que uses la IA como **consultor de arquitectura**, no como generador de código. Este ejercicio es de **pensar y decidir**, no de pegar. El entregable principal es un **documento de análisis** + cambios acotados, no un montón de código nuevo.

El problema

El proyecto **ya es un caso de estudio de arquitectura**. La carpeta `documents/` incluye un documento, [server-capas-explicacion.md](#), que explica en detalle cómo está armada la arquitectura en capas (`controller` → `service` → `repository` → `DbPg`) y por qué se tomó cada decisión de diseño (`Pool` vs `Client`, queries parametrizadas, `dotenv`, etc.). Tu trabajo es analizar el código de `src/` con criterio, usando ese documento como referencia.

Cosas para mirar con ojo crítico:

- El `service` de `courses` es un pass-through (no agrega nada). El de `students` sí (calcula edad). ¿Tiene sentido tener una capa `service` que a veces no hace nada?
 - El acceso a datos (`DbPg`) **se traga los errores** (`catch` → `LogHelper.logError` → devuelve `null / 0`), así que ni el `repository` ni el `service` se enteran de si falló la base o si simplemente no había datos. ¿Está bien?
 - Cada `repository` hace `this.db = new DbPg()` en su constructor → cada uno tiene **su propia instancia de `DbPg`** y, por lo tanto, **su propio `Pool`** de conexiones. Con 2 entidades son 2 pools; con 10, 10 pools. ¿Conviene? ¿O debería haber un solo `DbPg / Pool` compartido por toda la app?
 - `entities/` (`Alumno`, `Curso`) casi no se usan. ¿Para qué están? ¿Deberían usarse más?
 - Hay `console.log` de debug en casi todas las capas (`controllers`, `services`, `repositories`, `db-config`). ¿Está bien para producción? ¿No debería ir todo por `LogHelper`?
-

Qué tenés que lograr

1. Un **documento** `prompting/entregas/06-analisis-arquitectura.md` con:

- Diagrama o descripción del flujo actual de una request (de la URL a la base y de vuelta).
 - **3 a 5 problemas o decisiones discutibles** que detectaste, con su impacto.
 - Para cada uno: una recomendación y su **trade-off** (qué ganás, qué perdés).
2. Implementar **una** de tus recomendaciones (la que más valor aporte), acotada y justificada.

Cómo encarar el prompting

 Este es el ejercicio donde la **iteración** y el **rol** brillan. Le estás pidiendo opinión experta, no código.

Buen prompt de arranque:

"Actuá como un arquitecto de software revisando un proyecto educativo. Te paso la estructura de carpetas y 3 archivos clave. Quiero que evalúes la separación en capas: ¿la capa service aporta valor en todos los casos? ¿el manejo de errores en el repository es adecuado? Dame un análisis crítico con trade-offs, no me reescribas el código todavía."

 **Trampa:** la IA es **aduladora**. Si le preguntás "¿está bien mi arquitectura?", te va a decir que sí. Pedile explícitamente que sea **crítica** y que asuma que algo está mal: "asumí que hay al menos 3 problemas de diseño y encontralos".

 **Otra trampa:** la IA te va a querer vender patrones grandilocuentes (Clean Architecture, DDD, CQRS, inyección de dependencias con contenedor). Para un CRUD de 2 tablas, eso es **sobre-ingeniería**. Pedile que las recomendaciones sean **proporcionales al tamaño del proyecto**.

 **Iteración clave:** cuando te proponga algo, repreguntá "¿qué problema concreto resuelve esto en ESTE proyecto? Dame un ejemplo de un bug que se evitaría." Si no puede, probablemente no lo necesitás.

Verificación del resultado

- ☐ El documento describe el flujo real de una request (verificable contra el código).
- ☐ Cada problema detectado **existe de verdad** en el código (no es genérico de "cualquier proyecto Node").
- ☐ Cada recomendación tiene un **trade-off explícito** (no solo ventajas).
- ☐ El cambio que implementaste **no rompe** los endpoints existentes.
- ☐ No metiste sobre-ingeniería: cada pieza nueva se justifica por un problema concreto.

 Pregunta para el oral: *de los problemas que encontró la IA, ¿con cuál NO estuviste de acuerdo y por qué?* (Si estuviste de acuerdo con todo, no analizaste, copiaste.)

Entrega

Bitácora + el documento de análisis + commit del cambio implementado. Este ejercicio se evalúa más por el **razonamiento** que por las líneas de código.